

Application Development DICT312

Jakarta Servlet

Dr Olalekan Samuel Ogunleye

School of Computing and Mathematical Science

University of Mpumalanga

Telephone: 013 002 0227

Office: Building 5 (Room 212)

Consultation(By Appointment Only)

Jakarta Servlet

Dr Olalekan Samuel Ogunleye
School of Computing and Mathematical Science
University of Mpumalanga

Objectives of this Session

By the end of this session, you should be able to:

Understanding what is Jakarta Servlet

Understand how to create a Jakarta Servlet

Understand how to filter Jakarta Request and Response

Understand how to perform Asynchronous Process in Jakarta

Understand how to use Nonblocking I/O in Jakarta Servlet

Understand how to use Server Push in Jakarta Servlet

Introduction

In this session, the concept Jakarta Servlet, and how you can create a Servlet class.

Also, I will how we can filter the requests and responses, what is the Forward and RequestDispatcher feature and how we can use them, how we can work with the asynchronous process in a Servlet, how we can work with Nonblocking I/O, and how we can use the Server Push.

This session is very practical, and after we are done with this session, you will be able to create Servlets in your Jakarta EE applications using the best practices to it.

Understanding Jakarta Servlet

The technology that provides support to the network communication in a request response was is Jakarta Servlet.

This means that the Servlet allows communication where a requester applies a request to the server and the server sends back a response.

The Jakarta servlet supports any protocol. However, the servlet specification requires the HTTP support because the HTTP is the most commonly used protocol.

Understanding Jakarta Servlet

The technology that provides support to the network communication in a request response was is Jakarta Servlet.

This means that the Servlet allows communication where a requester applies a request to the server and the server sends back a response.

The Jakarta servlet supports any protocol. However, the servlet specification requires the HTTP support because the HTTP is the most commonly used protocol.

Understanding Jakarta Servlet

In the Jakarta Servlet, you will create a Servlet class that is responsible to process the requests and send a response to the requester.

Each Servlet class is responsible to process the request according to the listener configuration.

A listener configuration can be provided by annotation or by deployment descriptor (i.e web.xml).

Understanding Jakarta Servlet

A Servlet class extends the `jakarta.servlet.http.HttpServlet` class.

It has the methods to each HTTP method, and we can override them when we want to insert a behavior to a respective HTTP method request.

Understanding Jakarta Servlet

The following are the methods and the parameters

`doGet(HttpServletRequest req, HttpServletResponse resp)`

`doHead(HttpServletRequest req, HttpServletResponse resp)`

`doPost(HttpServletRequest req, HttpServletResponse resp)`

`doPut(HttpServletRequest req, HttpServletResponse resp)`

`doDelete(HttpServletRequest req, HttpServletResponse resp)`

`doOptions(HttpServletRequest req, HttpServletResponse resp)`

`doTrace(HttpServletRequest req, HttpServletResponse resp)`

Understanding Jakarta Servlet

The following are the methods and the parameters

doGet(Http)
doHead(F
doPost(Ht
doPut(Htt
doDelete(
doOptions
doTrace(F



IN ADDITION, THE SERVLET CLASS CONTAINS A SERVICE METHOD THAT, DEPENDING ON THE HTTP METHOD OF THE REQUEST, IS CALLED WHEN A REQUEST IS RECEIVED. THIS METHOD CALLS ONE OF THE PREVIOUSLY MENTIONED METHODS. USUALLY, WE OVERRIDE THE SERVICE METHOD AND WRITE THE LOGIC CODE THERE WHEN WE WRITE A SINGLE FUNCTION THAT WILL RUN INDEPENDENTLY OF THE HTTP METHOD IN ALL REQUESTS. WE WILL SOON SEE IT.

Jakarta Servlet Lifecycle

A Servlet has a well-defined life cycle that is managed by the Jakarta EE container.

Therefore, you (the developer) do not need to care about the life cycle.

Besides that, the developer can write some actions or behaviours to be executed in each step.

When a request is mapped to some Servlet, the Jakarta EE container executes the following steps. See the next Slide.

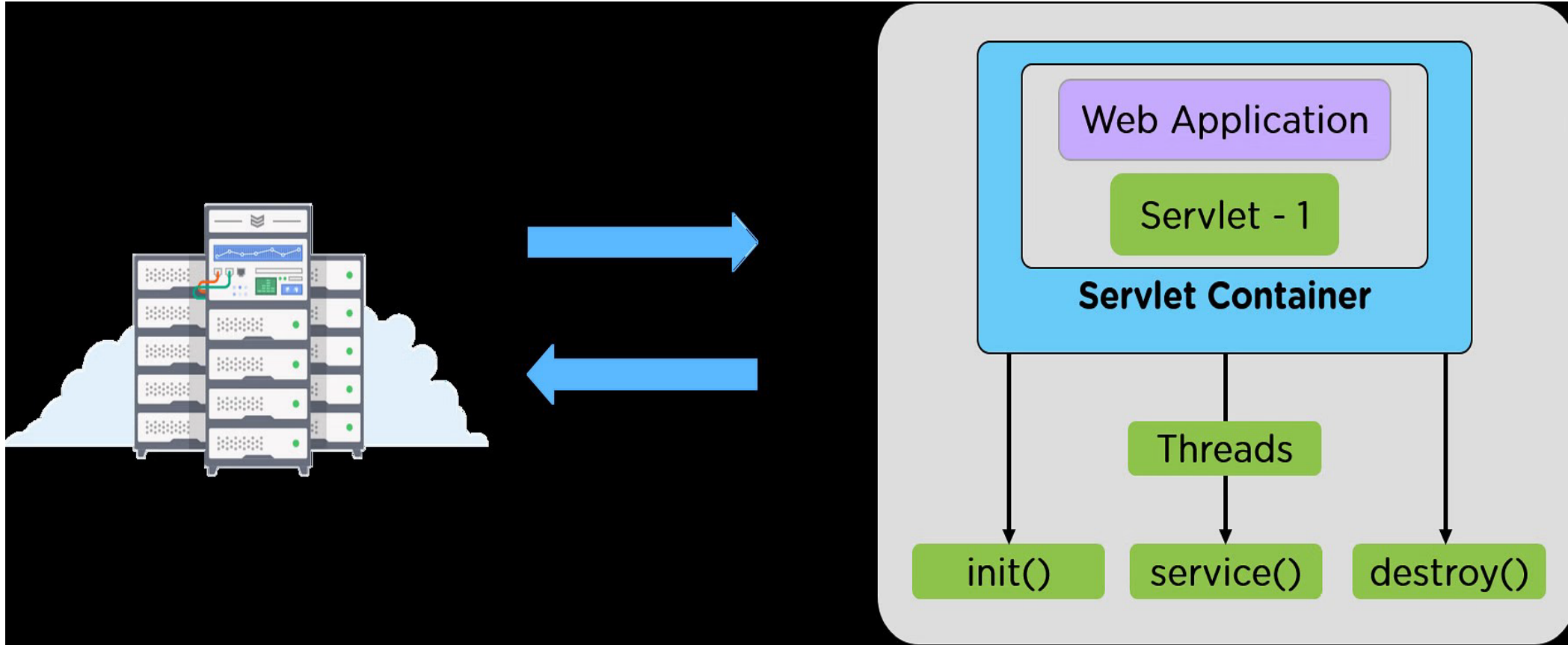
Jakarta Servlet Lifecycle

1. If an instance of the servlet class does not exist, then the container creates the instance. The servlet can be started either when the container is started or when the container detects that the servlet is needed.
2. The servlet is initialized by calling the init method. In case the developers want to insert some actions or behaviors at initialization time, he can override the init method.
3. After the servlet was initialized, the servlet is able to treat the requests. When a request is sent, the service method is called which is responsible for handling the request. The service method receives the ServletRequest and ServletResponse instances that represent the request and response respectively.

Jakarta Servlet Lifecycle

3. In a HttpServlet, the service method delegates the execution to a HTTP method (doGet, doPost, or others described earlier) according to the HTTP protocol. If the developer wants to write one action for all the HTTP methods, he can override the service method.
4. When the servlet is removed, the destroy method is called. Thus, if the developer wants to write one action to be executed at the remove time, he can override the destroy method.

Jakarta Servlet Architecture



Jakarta Servlet Architecture

```
import jakarta.servlet.*;
```

Once the Servlet interface is imported, and we inherit the HTTP Class, we begin with the Java Servlet's life cycle.

Jakarta Servlet Architecture

In the life cycle of a servlet, we have mainly three stages, which are mentioned below.

`init()`

`service()`

`destroy()`

We call these methods at their respective stages. The methods are resolved by generating the essential threads for the process to get executed.

Jakarta Servlet Architecture

1. init()

The `init()` is the germinating stage of any Java Servlet. When a URL specific to a particular servlet is triggered, the `init()` method is invoked.

Another scenario when the `init()` method gets invoked is when the servers are fired up. With every server starting up, the corresponding servlets also get started, and so does the `init()` method.

One important specialty of the `init()` method is the `init()` method only gets invoked once in the entire life cycle of the Servlet, and the `init()` method will not respond to any of the user's commands.

Jakarta Servlet Architecture

The init() method Syntax:

```
public void init() throws ServletException {  
    //init() method initializing  
  
}
```

Jakarta Servlet Architecture

2. Service()

The service() method is the heart of the life cycle of a Java Servlet. Right after the Servlet's initialization, it encounters the service requests from the client end.

The client may request various services like:

GET

PUT

UPDATE

DELETE

Jakarta Servlet Architecture

2. Service()

The service() method takes responsibility to check the type of request received from the client and respond accordingly by generating a new thread or a set of threads per the requirement and implementing the operation through the following methods.

doGet() for GET

doPut() for PUT

doUpdate() for UPDATE

doDelete() for DELETE

Jakarta Servlet Architecture

2. Service()

The service() method Syntax:

```
public void service(ServletRequest request, ServletResponse response)  
throws ServletException, IOException {  
  
}
```

Jakarta Servlet Architecture

3. destroy()

Like the `init()` method, the `destroy()` method is also called only once in the Java Servlet's entire life cycle.

When the `destroy()` method is called, the Servlet performs the cleanup activities like:

- Halting the current or background threads
- Making a recovery list of any related data like cookies to Disk.

After that, the Servlet is badged, ready for the Garbage collector to have it cleared.

Jakarta Servlet Architecture

3. destroy()

The destroy() method Syntax:

```
public void destroy() {  
    //destroy() method finalizing  
  
}
```

Jakarta Servlet Multithreading

The Servlet should be able to respond to multiple concurrent requests sent to them.

To achieve this, the Jakarta EE web container creates one thread for each request and these threads share the objects stored in the Servlet context, objects stored in the HTTP session, and any resource that is in the instance variables of the servlet.

The resources, objects, or values that are inside the request or response instances are not shared between the requests.

Jakarta Servlet Multithreading

You should pay attention to the questions about concurrency to avoid the side effects.

Therefore, if you create or access an instance variable of the servlet inside any of the HTTP methods of the servlet mentioned earlier, such as `doGet` and `doPost`, we must be aware that the very same servlet instance is reused between all the requests hitting the servlet.

As a tip for the best practice, avoid assigning the request or session scoped variables as the instance variables of the servlet.

Creating a Jakarta Servlet

This will be done in our Lab Activities.

However, if you want to Start you can go ahead and practice.

Please note that we would be using Wildfly Application Server for.

Check out this Video?

Configuring Jakarta Servlet using Annotation

How this is done will be done in the Lab Activities

However, before then, you need to learn about the following:

URL Patterns: The URL patterns defining each URL will be executed by the servlet. If a URL matches the servlet's URL patterns, then this Servlet will process the request.

Request Parameter: The request parameters are string values sent from the client-side to the servlet. These parameters can be sent by both, the URI query string and the POST-ed data. The request parameters are stored as a map of the name-value pairs inside the `HttpServletRequest` object. And you can handle it by the following methods:

Configuring Jakarta Servlet using Annotation

getParameter

getParameterNames

getParameterValues

getParameterMaps

How all these are used will be shown in the lab activities.

HTTP Headers

The HTTP has the HTTP headers that can be sent from the client to the server on the HTTP request or can be sent from the server to the client on the HTTP response. To access the headers on an HTTP request, you can use the following methods:

`getHeader`

`getHeaders`

`getHeaderNames`

To get the HTTP header from a request, we use `HttpServletRequest` and to set HTTP header to the response, we use `HttpServletResponse`

How all these are used will be shown in the lab activities.

Managing Session

A common task when we are working with the HTTP protocol is to manage the session to put some data that will survive for the many requests.

Basically, when the client starts the access to any HTTP server, an HTTP session is created in the HTTP server; this session is kept for many or all requests done by the client.

The Servlet provides the `HttpSession` interface that is used to manage the HTTP session. So, you can invalidate a session to create another one or put some data to it.

Managing Session

Note that the HTTP session has a time out, and after a time out happens, the session is invalidated and a new session is created. The following are the main methods of the HttpSession interface:

getSessiongetId

getCreationTime

invalidate

getLastAccessedTime

setAttribute

getAttribute

Managing Session

To get the session associated to the request call, the following method of `HttpServletRequest` is used:

```
HttpSession session = req.getSession();
```

In the code snippet above, the current session will be returned or null in case there is no session associated.

If you want a new session to be created in case there is no session associated, you can call the following method of `HttpServletRequest`:

```
HttpSession session = req.getSession(true);
```

Managing Session

To put some data to session, you can use this method of
HttpSession:session.setAttribute("user", user);

In this code snippet above, we are setting the user as an attribute and it is associated with the “user” key.

So, to get the user from the session, you can call the following method from
HttpSession:session.getAttribute("user");

This method returns an attribute from the session according to some key.

Filtering the Request and Response Type

Many a times, we want to apply some transformations in the data and headers of an HTTP request or HTTP response and/or make any validation before the request is executed.

A good approach to do it is using the filters, where we filter the requests and apply the transformations.

Using this approach, you can decouple the request processing logic from the transformation logic, and reuse the transformation logic in many servlets.

The following are some scenarios that we can be filters:

Filtering the Request and Response Type

Logging and auditing

Authentication

Tokenizing

Data compression

Image conversion

Decrypting data

It is nice! But how can we use this approach in a servlet? The Jakarta Servlet has a feature to create the filter classes to intercept the requests and responses and apply some logic to this. Basically, a filter class is a class that implements the Filter interface and uses the FilterChain and FilterConfig to handle how the filter should work. The Servlet Filters is based on the pattern called Chain of Responsibility, that is a behavioral pattern of Gang of Four (GoF).

Understanding Forward and Request Dispatcher

The RequestDispatcher is a useful feature that provides support to forward or include another resource to make a response.

Basically the Servlet has an object from RequestDispatcher and it has the methods, **include** and **forward**, to call these functionalities:

Include: The include is used when we want to include another web component in the response, but the control of the processing still is of the servlet that makes the include

When we apply a forward or include, the attributes of the request is shared between the web components. So, the attributes will be exactly the same.

Understanding Forward and Request Dispatcher

The RequestDispatcher is a useful feature that provides support to forward or include another resource to make a response.

Basically the Servlet has an object from RequestDispatcher and it has the methods, **include** and **forward**, to call these functionalities:

Forward: The forward is used when we want to transfer the control of the processing to another web component to generate the response.

In other words, the responsibility to generate a response is of the servlet forwarded.

Example of this in the lab.

Asynchronous Processing

In the Jakarta EE platform, each request done to a Servlet is processed by a thread.

These threads stay in a pool of container threads and after processing a request, it is returned to the pool of container threads.

With this, the request processing needs to be very fast to release the thread to another request processing.

It is very important, because it can boost or degraded the application performance a lot according to the use of these threads.

Asynchronous Processing

But sometimes we need a long-term process to execute some kind of hard process.

In these cases, we can use the asynchronous processing disposed on the Jakarta Servlet

The asynchronous processing in the servlet permits us to delegate the execution to an asynchronous process managed by the context that will generate the response to the client.

Asynchronous Processing

Note that the client will keep waiting to respond, but the difference is that the execution will happen in another thread, permitting the servlet thread to be released to execute another request

To see how this works, we will create a ReportServlet that will emulate the behaviour of creating reports. See this in the lab activities.

Using Nonblocking I/O

Asynchronous processing is a good servlet feature to improve the performance of the servlet threads and permit a thread to be available to respond to the other requests.

But, imagine you are receiving a request and the network is very slow. The receiver will read the data faster than the requester can write and the result will be a momentarily sitting idle of the receiver.

Therefore, even with the asynchronous processing, the servlet thread will be kept for a long time and will decrease the performance.

Using Nonblocking I/O

Note that we are talking about the input/ output stream between the client and the server.

To solve this problem, the servlet has the Nonblocking I/O feature that permits reading and writing the data asynchronously to a listener when the data is available.

This mechanism creates a separate thread-safe to read and write the data. As a result, the servlet thread responsible for receiving the requests won't wait for the read/write to finish to be available for the other requests.

Using Nonblocking I/O

To use the Nonblocking I/O, you should enable the async request processing in the servlets and filters, like in the asynchronous processing.

Using Server Push

To use the Nonblocking I/O, you should enable the async request processing in the servlets and filters, like in the asynchronous processing.

Conclusion

The Jakarta Servlet is a good specification to permit the applications to communicate using many protocols, but mainly the HTTP protocol.

This specification is very important, because many other frameworks came based on it to make the communication using the HTTP protocol.

As an example of the frameworks that used this specification, we can quote Spring, Struts, VRaptor, and the others.

In the Jakarta Servlet you can use many interesting features described throughout this session, such as, Filters, Asynchronous Processing, NonBlocking/IO, and Server Push.

Conclusion

You are now able to use the principal features of Jakarta Servlet in your project to promote the performance solutions to it.

In our next class, we will explore the Jakarta Context Dependency Injection Specification and learn how to use it using the best Practice.



UNIVERSITY OF
MPUMALANGA

Thank you